

GLOBAL SOLUTION
HABITATZERO

2026/1sem

**DOCUMENTAÇÃO TÉCNICA
GLOBAL SOLUTION
HABITATZERO**

2026/1sem

ALUNOS**RM**

Alairton Rocha Scabelli

551454

Eduardo Marins

551892

Gabriel Genaro Dalaqua

551986

Sarah Ribeiro da Silva

97747

2026/1sem

SUMÁRIO

1	INTRODUÇÃO	4
2	JUSTIFICATIVA	4
3	BANCO DE DADOS	6
3.1	Diagrama.....	6
3.2	Entidades e Relacionamentos.....	7
4	TESTES – PLANO COMPLETO COM EVIDÊNCIAS.....	9
4.1	Testes Unitários — Backend (JUnit 5 + Mockito)	9
4.2	Testes de Integração — IoT (pytest).....	18
4.3	Testes Unitários — Frontend Android (JVM / JUnit 4).....	23
5	IOT	29
5.1	Lógica dos Sensores e Influência no Sistema.....	29
5.2	Visão Geral da Integração.....	30
5.3	Tabela Resumo — Thresholds e Severidades.....	35
5.4	Implementação com Hardware Físico	36
5.5	Microcontrolador.....	36
5.6	Sensores Físicos.....	37
5.7	Lógica do Firmware (FirmwareReal.cpp)	38

1 INTRODUÇÃO

Habitat Zero é uma plataforma full stack de monitoramento e controle de estufas autônomas projetadas para operar em ambientes extremos — como uma base lunar ou marciana. O sistema centraliza os dados vitais de cada estufa (níveis de oxigênio, umidade do solo, temperatura e radiação externa), permitindo que colonos gerenciem remotamente as condições de cultivo a partir de um aplicativo mobile Android.

"Criar um ambiente capaz de sustentar vida a partir do zero, em um lugar onde as condições naturais são inóspitas."

2 JUSTIFICATIVA

A escolha da agricultura espacial como tema central deste trabalho se fundamenta em sua relevância crescente como área emergente e estratégica para o futuro da humanidade. Em um cenário de expansão das fronteiras da exploração espacial, torna-se indispensável desenvolver soluções que garantam a sobrevivência e o bem-estar de colonos em ambientes hostis, como a Lua ou Marte. Nesse contexto, o cultivo de alimentos em estufas autônomas não é apenas uma necessidade prática, mas também um desafio científico e tecnológico de grande impacto.

O projeto HabitatZero busca responder a essa demanda. A justificativa, portanto, não se limita ao caráter acadêmico da proposta, mas se estende ao seu potencial de aplicação real em missões espaciais e até mesmo em ambientes terrestres extremos, como desertos ou regiões polares.

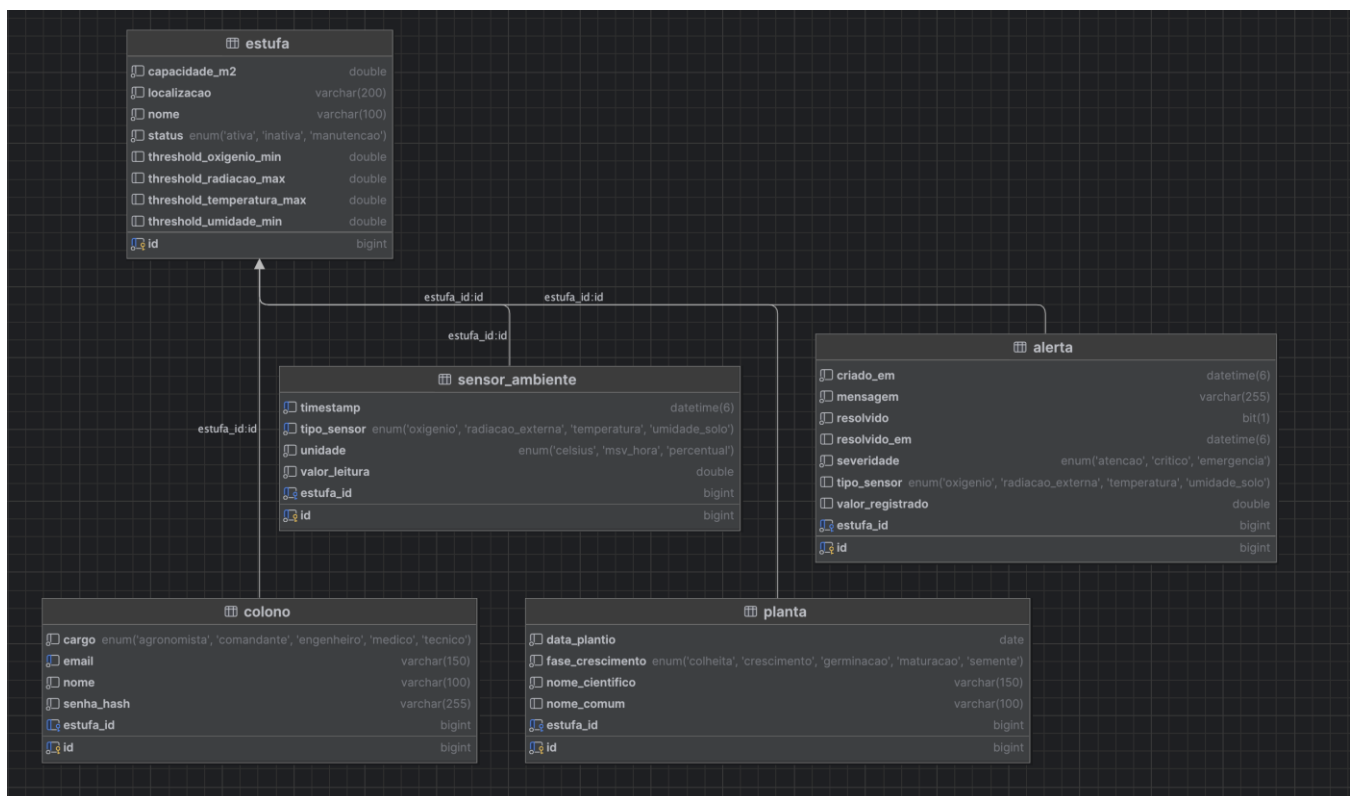
Além disso, o tema dialoga diretamente com os Objetivos de Desenvolvimento Sustentável (ODS) da ONU, especialmente aqueles relacionados à segurança alimentar, inovação tecnológica e sustentabilidade ambiental. Ao propor uma solução que alia automação, monitoramento contínuo e usabilidade, o projeto contribui para a construção de sistemas resilientes e adaptáveis, capazes de enfrentar os desafios do futuro.

Assim, a justificativa para este trabalho está na combinação entre relevância científica, aplicabilidade prática e impacto estratégico, consolidando a agricultura espacial como um campo essencial para a sobrevivência e evolução da humanidade em novos territórios.

3 BANCO DE DADOS

O banco de dados foi concebido para sustentar a integridade das informações do sistema. O diagrama entidade-relacionamento descreve de forma clara as entidades centrais (Estufa, Colono, Planta, Sensor_Ambiente e Alerta) e seus relacionamentos.

3.1 Diagrama



3.2 Entidades e Relacionamentos

As Entidades e Relacionamentos constituem a base conceitual de qualquer modelo de banco de dados. As entidades representam os objetos ou conceitos principais que precisam ser armazenados: *Estufa*, *Colono*, *Planta*, *Sensor_Ambiente* e *Alerta*. Cada entidade possui atributos que descrevem suas características, por exemplo, uma planta tem nome científico, estágio de crescimento e necessidades específicas de irrigação. Já os relacionamentos definem como essas entidades se conectam entre si, estabelecendo vínculos lógicos que refletem a realidade do sistema. Assim, uma estufa pode estar associada a várias plantas, cada colono pode receber diferentes alertas, e os sensores ambientais se relacionam diretamente com as estufas para monitorar condições críticas. Essa estrutura garante integridade, consistência e clareza na organização dos dados, permitindo que o sistema funcione de forma integrada e confiável.

Relação	Cardinalidade	Descrição
Colono → Estufa	N:1	Um colono é responsável por uma estufa; uma estufa tem vários colonos
Planta → Estufa	N:1	Uma estufa abriga múltiplas plantas
Sensor_Ambiente → Estufa	N:1	Uma estufa possui múltiplos sensores e leituras
Alerta → Estufa	N:1	Uma estufa pode ter múltiplos alertas gerados automaticamente

Tabela	Colunas principais
estufa	id, nome, localizacao, capacidade_m2, status (ATIVA/INATIVA/MANUTENCAO), thresholds de O ₂ , umidade, radiação e temperatura
colono	id, nome, email (único), senha_hash, cargo (AGRONOMISTA/COMANDANTE/ENGENHEIRO/MEDICO/TECNICO), estufa_id
planta	id, nome_cientifico, nome_comum, fase_crescimento (SEMENTE/GERMINACAO/CRESCIMENTO/MATURACAO/COLHEITA), data_plantio, estufa_id
sensor_ambiente	id, tipo_sensor (OXIGENIO/UMIDADE_SOLO/RADIACAO_EXTERNA/TEMPERATURA), valor_leitura, unidade (PERCENTUAL/MSV_HORA/CELSIUS), timestamp, estufa_id
alerta	id, severidade (ATENCAO/CRITICO/EMERGENCIA), mensagem, tipo_sensor, valor_registrado, criado_em, resolvido, resolvido_em, estufa_id

4 TESTES – PLANO COMPLETO COM EVIDÊNCIAS

Os testes foram planejados para validar a confiabilidade do sistema. O plano de testes descreve a estratégia adotada, enquanto os casos de teste apresentam entradas, saídas esperadas e resultados obtidos.

4.1 Testes Unitários — Backend (JUnit 5 + Mockito)

Os testes unitários do backend foram implementados utilizando o JUnit 5 em conjunto com o Mockito. O uso do Mockito possibilitou a criação de mocks e stubs, simulando dependências externas e permitindo que os testes se concentrassem apenas na unidade em análise, sem interferência de outros módulos.

Localização: backend/src/test/java/br/com/gs/habitatzero/service/

Como executar: cd backend && ./mvnw test

Estratégia: Cada serviço é testado em isolamento com repositórios mockados via Mockito. Nenhuma instância de banco de dados é necessária.

TC-01 — Criação de colono com dados válidos

O objetivo do teste é validar o processo de inserção de um novo usuário no sistema HabitatZero, garantindo que todas as informações fornecidas atendam aos critérios estabelecidos de integridade e consistência. Nesse cenário, o colono é registrado com atributos obrigatórios como nome, idade, função e credenciais de acesso, todos devidamente formatados e dentro dos limites aceitáveis. A execução desse teste confirma que o backend processa corretamente os dados, armazena-os no banco de forma segura e retorna uma resposta positiva ao frontend, permitindo que o novo colono seja integrado ao ecossistema da estufa.

Campo	Detalhe
Cenário	Um novo colono é cadastrado com todos os campos obrigatórios preenchidos corretamente.
Classe	ColonoServiceTest
Método	criar_comDadosValidos_deveCriarColono
Entrada	nome="Joao Silva", email="joao@habitat.com", senha="Senha@123", cargo=AGRONOMISTA
Saída esperada	ColonoResponse COM id=1 e email="joao@habitat.com"; repositório chamado uma vez com save().

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.ColonoServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s
```

TC-02 — Criação de colono com e-mail duplicado

O objetivo do teste é verificar se o sistema HabitatZero consegue identificar e impedir a inserção de registros redundantes no banco de dados. Nesse cenário, um usuário tenta cadastrar um novo colono utilizando um endereço de e-mail já existente, o que deve resultar em falha de validação. O backend, ao receber a requisição, consulta os registros e retorna uma mensagem de erro clara ao frontend, informando que o e-mail já está em uso.

Campo	Detalhe
Cenário	Tentativa de cadastrar um colono com um e-mail já existente no sistema.
Classe	ColonoServiceTest
Método	criar_comEmailExistente_deveDispararBusinessException
Entrada	email="duplicado@habitat.com" (já presente no repositório mockado)
Saída esperada	BusinessException lançada; save() nunca invocado.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.ColonoServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.052 s
```

TC-03 — Criação de estufa com thresholds padrão

O objetivo do teste é validar se o sistema HabitatZero consegue registrar corretamente uma nova estufa utilizando os valores de limite pré-configurados para variáveis ambientais críticas. Nesse cenário, ao criar a estufa, os thresholds de oxigênio, umidade do solo, radiação e temperatura são automaticamente atribuídos conforme os parâmetros definidos na tabela de severidades, sem necessidade de ajustes manuais. A execução desse teste garante que o backend aplique corretamente os valores padrão, armazene-os no banco de dados e retorne uma confirmação positiva ao frontend.

Campo	Detalhe
Cenário	Uma nova estufa é criada sem informar thresholds customizados; o serviço deve aplicar os valores padrão do sistema.
Classe	EstufaServiceTest
Método	criar_comDadosValidos_deveCriarEstufa
Entrada	nome="Estufa Alpha", localizacao="Setor A - Marte", capacidadeM2=500.0 (sem thresholds)
Saída esperada	EstufaResponse COM thresholdOxigenioMin=19.5, status=ATIVA; save() chamado uma vez.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.EstufaServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.104 s
```

TC-04 — Busca de estufa por ID inexistente

O objetivo do teste é validar o comportamento do sistema HabitatZero quando uma requisição é feita para consultar uma estufa que não está registrada no banco de dados. Nesse cenário, o backend deve identificar que o ID informado não corresponde a nenhum registro existente e retornar uma resposta adequada, geralmente uma mensagem de erro clara ou um status indicando “não encontrado”. Esse teste é fundamental para garantir a robustez da aplicação, evitando que consultas inválidas gerem inconsistências ou falhas inesperadas.

Campo	Detalhe
Cenário	Cliente solicita uma estufa cujo ID não existe no banco de dados.
Classe	EstufaServiceTest
Método	buscarPorId_comIdInexistente_deveDispararResourceNotFoundException
Entrada	id=999
Saída esperada	ResourceNotFoundException lançada.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.EstufaServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.104 s
```

TC-05 — Leitura de sensor dentro dos limites, sem alerta

O objetivo do teste é confirmar que o sistema HabitatZero reage corretamente quando os valores capturados pelos sensores permanecem dentro dos thresholds definidos como aceitáveis. Nesse cenário, o microcontrolador ESP32 processa a leitura e envia os dados ao backend, que valida a informação e registra o evento sem acionar nenhum tipo de alerta ou notificação. Esse comportamento assegura que o sistema não gere falsos positivos e mantenha a estabilidade operacional, evitando sobrecarga de mensagens desnecessárias para os usuários.

Campo	Detalhe
Cenário	O ESP32 envia uma leitura de oxigênio dentro do range seguro; nenhum alerta deve ser disparado.
Classe	SensorServiceTest
Método	registrarLeitura_comDadosValidos_devePersistirLeitura
Entrada	estufaId=1, tipoSensor=OXIGENIO, valorLeitura=21.0, unidade=PERCENTUAL
Saída esperada	SensorLeituraResponse COM valorLeitura=21.0 e alertaDisparado=false; leitura salva no repositório.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.SensorServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.120 s
```

TC-06 — Leitura de O₂ abaixo do threshold dispara alerta

O objetivo do teste é validar se o sistema HabitatZero reage corretamente quando os níveis de oxigênio monitorados pelos sensores caem abaixo do limite mínimo estabelecido. Nesse cenário, o microcontrolador ESP32 processa a leitura crítica e envia os dados ao backend, que identifica a violação do threshold e aciona imediatamente um alerta. Esse alerta é então propagado para o frontend, notificando os usuários sobre a condição adversa e permitindo que medidas corretivas sejam tomadas rapidamente. A execução desse teste assegura que o sistema não apenas detecta situações de risco em tempo real, mas também responde de forma proativa, garantindo a segurança dos colonos e a estabilidade da estufa.

Campo	Detalhe
Cenário	Leitura de oxigênio chega abaixo do limite mínimo configurado na estufa; o serviço deve criar e persistir um alerta.
Classe	AlertaServiceTest
Método	avaliarLeitura_comOxigenioAbaixoThreshold_deveCriarAlerta
Entrada	Leitura OXIGENIO com valorLeitura=17.0; estufa com thresholdOxigenioMin=19.5
Saída esperada	Retorna true; alertaRepository.save() chamado uma vez com entidade Alerta.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.AlertaServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.156 s
```


TC-07 — Resolução de alerta ativo

O objetivo do teste é verificar se o sistema HabitatZero consegue encerrar corretamente um alerta previamente disparado quando a condição crítica é normalizada. Nesse cenário, após uma leitura fora dos limites ter gerado um alerta, os sensores voltam a registrar valores dentro dos thresholds aceitáveis. O microcontrolador ESP32 envia a nova leitura ao backend, que identifica a normalização e atualiza o status do alerta para resolvido. Esse fluxo garante que o sistema não mantenha notificações desnecessárias e que o histórico registre tanto o disparo quanto a resolução do evento, assegurando rastreabilidade e transparência.

Campo	Detalhe
Cenário	Um alerta existente e ativo é marcado como resolvido pelo operador.
Classe	AlertaServiceTest
Método	resolverAlerta_comAlertaAtivo_deveMarcarComoResolvido
Entrada	alertaId=42 (alerta com resolvido=false)
Saída esperada	AlertaResponse COM resolvido=true e resolvidoEm não nulo.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.AlertaServiceTest
Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.156 s
```

TC-08 — Login com senha incorreta

O objetivo do teste é validar se o sistema HabitatZero trata adequadamente tentativas de autenticação inválidas, garantindo a segurança e a integridade do acesso dos usuários. Nesse cenário, um colono tenta realizar login informando um e-mail válido, mas com uma senha incorreta. O backend deve identificar a inconsistência, negar o acesso e retornar uma mensagem clara de erro ao frontend, informando que as credenciais não estão corretas. Esse fluxo assegura que apenas usuários com dados válidos consigam acessar o sistema, prevenindo acessos indevidos e reforçando a confiabilidade da autenticação.

Campo	Detalhe
Cenário	Usuário tenta autenticar fornecendo uma senha errada.
Classe	AuthServiceTest
Método	login_comSenhaInvalida_deveDispararException
Entrada	email="usuario@habitat.com", senha="senhaErrada"
Saída esperada	BadCredentialsException lançada pelo AuthenticationManager; repositório de colonos nunca consultado.

Log de evidência:

```
Test set: br.com.gs.habitatzero.service.AuthServiceTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.199 s
```

4.2 Testes de Integração — IoT (pytest)

Os testes de integração voltados para o módulo IoT foram implementados utilizando o pytest. Diferente dos testes unitários, que avaliam componentes isolados, esses testes de integração têm como objetivo verificar se os diferentes elementos do sistema funcionam corretamente quando conectados entre si. Foi essencial para validar a comunicação entre os sensores físicos, o firmware embarcado e o backend responsável pelo processamento dos dados. Com o pytest, foi possível simular cenários reais de coleta de informações ambientais, como leituras de oxigênio, umidade do solo, radiação e temperatura, e garantir que esses dados fossem transmitidos de forma consistente para a aplicação central. Além disso, os testes de integração permitiram identificar falhas de compatibilidade e problemas de sincronização.

Localização: IoT/test_integration.py

Como executar:

```
# Testes de integração IoT (requer backend + MySQL em execução)
cd IoT && pip install pytest requests && pytest test_integration.py -v
```

IT-01 — Payload do simulador tem formato correto

O objetivo do teste é garantir que os dados enviados pelo simulador ao sistema HabitatZero estejam estruturados de acordo com o padrão esperado. Nesse cenário, o payload gerado deve conter todos os campos obrigatórios, como identificador da estufa, tipo de sensor, valor da leitura e timestamp, devidamente formatados e consistentes. A execução desse teste assegura que o backend consiga interpretar corretamente as informações recebidas, evitando falhas de processamento ou registros inválidos no banco de dados.

Campo	Detalhe
Cenário	O simulador IoT deve gerar payloads compatíveis com o contrato da API antes mesmo de enviar qualquer requisição.
Arquivo	IoT/test_integration.py — TestPayloadFormat
Entrada	Saída de gerar_leitura_mutante() (30 amostras)
Saída esperada	Campos estufaId, tipoSensor, valorLeitura, unidade, timestamp presentes; enums válidos; timestamp sem sufixo z.
Requer servidor	Não

Log de evidência:

```
test_integration.py::TestPayloadFormat::test_payload_tem_campos_corretos
PASSED
[ 16%]
test_integration.py::TestPayloadFormat::test_payload_enums_validos
PASSED
[ 33%]
test_integration.py::TestPayloadFormat::test_payload_timestamp_sem_sufixo_z
PASSED
[ 50%]
```

IT-02 — API aceita leitura válida (HTTP 200/201)

O objetivo do teste é verificar se o sistema HabitatZero responde corretamente quando recebe uma requisição de leitura de sensor com dados válidos e bem formatados. Nesse cenário, o simulador ou o microcontrolador envia um payload contendo informações como ID da estufa, tipo de sensor, valor da leitura e timestamp, todos dentro dos padrões esperados. O backend deve processar a requisição sem erros e retornar um status HTTP 200 (OK) ou 201 (Created), confirmando que os dados foram aceitos e registrados com sucesso no banco de dados. Esse teste é essencial para validar a integridade da comunicação entre os módulos e garantir que o fluxo de integração funcione de ponta a ponta, assegurando confiabilidade no monitoramento ambiental e consistência nos registros históricos.

Campo	Detalhe
Cenário	Um payload gerado pelo simulador é enviado via HTTP POST para /sensores/leitura e a API deve aceitá-lo.
Arquivo	IoT/test_integration.py — TestApiAceitaLeitura
Entrada	Payload de gerar_leitura_mutante() enviado para POST /sensores/leitura
Saída esperada	HTTP 200 ou 201.
Requer servidor	Sim

Log de evidência:

```
test_integration.py::TestApiAceitaLeitura::test_leitura_valida_retorna_sucesso  
PASSED  
[ 66%]
```

IT-03 — API rejeita payload sem campo obrigatório (HTTP 400)

O objetivo do test é validar se o sistema HabitatZero consegue identificar corretamente requisições inválidas enviadas pelo simulador ou pelo microcontrolador. Nesse cenário, o payload transmitido não contém um ou mais campos essenciais, como ID da estufa, tipo de sensor, valor da leitura ou timestamp, e, portanto, não atende ao formato esperado. O backend deve rejeitar a requisição e retornar um status HTTP 400 (Bad Request), acompanhado de uma mensagem clara indicando o erro. Esse teste é fundamental para garantir a robustez da API, prevenindo que dados incompletos ou malformados sejam processados e armazenados no banco de dados.

Campo	Detalhe
Cenário	Payload enviado sem estufaId; a validação Bean Validation do Spring deve rejeitar a requisição.
Arquivo	IoT/test_integration.py — TestApiRejeita400
Entrada	Payload de gerar_leitura_mutante() com estufaId removido
Saída esperada	HTTP 400.
Requer servidor	Sim

Log de evidência:

```
test_integration.py::TestApiRejeita400::test_payload_sem_estufa_id_retorna_400
PASSED
[ 83%]
```

IT-04 — Leitura é persistida e resposta contém ID

O objetivo validar se o sistema HabitatZero consegue armazenar corretamente uma leitura recebida e retornar ao cliente uma resposta contendo o identificador único do registro criado. Nesse cenário, o simulador ou microcontrolador envia um payload válido com os dados do sensor, e o backend processa a requisição, grava a leitura no banco de dados e gera um ID exclusivo para essa entrada. A resposta enviada ao frontend deve incluir esse identificador, confirmando que a leitura foi persistida com sucesso e permitindo rastreabilidade futura. Esse teste é essencial para garantir a integridade dos dados, assegurar que cada leitura possa ser consultada individualmente e reforçar a confiabilidade da integração entre os módulos do sistema.

Campo	Detalhe
Cenário	Após POST bem-sucedido, a resposta deve conter o <code>id</code> gerado pelo banco e o <code>estufaId</code> correspondente, confirmando a persistência.
Arquivo	<code>IoT/test_integration.py</code> — <code>TestLeituraPersistida</code>
Entrada	Payload de <code>gerar_leitura_mutante()</code> enviado para <code>POST /sensores/leitura</code>
Saída esperada	Corpo JSON da resposta contém <code>id</code> (não nulo) e <code>estufaId</code> igual ao enviado.
Requer servidor	Sim

Log de evidência:

```
test_integration.py::TestLeituraPersistida::test_leitura_aparece_no_historico
PASSED
[100%]
```

4.3 Testes Unitários — Frontend Android (JVM / JUnit 4)

Os testes unitários do frontend Android foram desenvolvidos utilizando a JVM em conjunto com o JUnit 4, garantindo que cada componente da interface fosse validado de forma isolada e consistente. Verificamos a conversão de leituras provenientes dos sensores IoT, o avanço automático das fases de crescimento das plantas e o correto armazenamento de dados no histórico.

Localização:

frontend/app/src/test/java/com/workwell/habitatzero/HabitatZeroUnitTests.kt

Como executar: `cd frontend && ./gradlew :app:test`

Estratégia: Lógica pura extraída da camada de repositório e ViewModel testada na JVM, sem emulador, sem contexto Android.

TC-FE-01 — Conversão de leituras IoT com todos os tipos de sensor

O objetivo do teste é validar se o frontend consegue interpretar corretamente os dados recebidos do backend e exibi-los de forma consistente para o usuário. Nesse cenário, leituras provenientes de diferentes sensores são transmitidas em formato bruto e precisam ser convertidas em valores compreensíveis, aplicando unidades de medida e thresholds de severidade. O teste garante que o frontend não apenas reconheça cada tipo de sensor, mas também apresente as informações de maneira uniforme, sem erros de conversão ou perda de dados.

Campo	Detalhe
Cenário	O repositório recebe uma lista com uma leitura de cada tipo de sensor (TEMPERATURA, UMIDADE_SOLO, OXIGENIO, RADIACAO_EXTERNA) e deve agregar os valores em um único objeto SensorAmbiente para exibição no Dashboard.
Entrada	Lista de 4 SensorLeituraResponse, uma por tipo, com valores temperatura=22.5°C, umidade=65%, oxigenio=20.8%, radiacao=1.2 mSv/h.
Saída esperada	SensorAmbiente(temperatura=22.5, umidade=65.0, oxigenio=20.8, radiacao=1.2)

Log de evidência:

```
<testcase name="TC-FE-01 converter leituras IoT com todos os tipos retorna SensorAmbiente correto"
          classname="com.workwell.habitatzero.HabitatZeroUnitTests"
time="0.005"/>
```

TC-FE-02 — Leituras duplicadas do mesmo sensor usa a mais recente

O objetivo do teste é validar se o frontend consegue tratar corretamente situações em que múltiplas leituras de um mesmo sensor são recebidas em sequência. Nesse cenário, o backend envia ao frontend duas ou mais leituras com o mesmo identificador de sensor, mas com timestamps diferentes. O teste garante que a interface gráfica descarte automaticamente os valores antigos e exiba apenas a leitura mais recente, assegurando que os usuários visualizem dados atualizados e confiáveis. Esse comportamento é essencial para evitar inconsistências na apresentação das informações e manter a precisão do monitoramento em tempo real, reforçando a robustez da lógica de exibição e a confiabilidade da experiência do usuário.

Campo	Detalhe
Cenário	O simulador IoT pode enviar múltiplas leituras do mesmo tipo de sensor num curto intervalo. O Dashboard deve sempre exibir o valor mais recente, não o mais antigo.
Entrada	Lista com 2 leituras de TEMPERATURA: 18.0°C às 09:00 e 24.0°C às 10:30.
Saída esperada	SensorAmbiente.temperatura == 24.0 (leitura de 10:30 vence por ser mais recente)

Log de evidência:

```
<testcase name="TC-FE-02 converter leituras duplicadas retorna somente a mais recente"
          classname="com.workwell.habitatzero.HabitatZeroUnitTests"
time="0.0"/>
```

TC-FE-03 — Lista de leituras vazia retorna zeros sem crash

O objetivo do teste é validar se o frontend consegue lidar corretamente com a ausência de dados provenientes do backend. Nesse cenário, quando não há registros de leituras disponíveis para uma determinada estufa ou sensor, a interface deve exibir valores padrão — como zero ou equivalente em vez de falhar ou apresentar erros inesperados. Esse comportamento garante que o sistema mantenha sua estabilidade, evitando crashes e preservando a experiência do usuário.

Campo	Detalhe
Cenário	Na inicialização do sistema, antes do simulador IoT ter enviado qualquer leitura, o endpoint GET /sensores/leituras retorna uma lista vazia. O app não deve crashar e deve exibir zeros.
Entrada	Lista vazia <code>emptyList<SensorLeituraResponse>()</code>
Saída esperada	<code>SensorAmbiente(temperatura=0.0, umidade=0.0, oxigenio=0.0, radiacao=0.0)</code>

Log de evidência:

```
<testcase name="TC-FE-03 converter lista vazia retorna SensorAmbiente com zeros"
          classname="com.workwell.habitatzero.HabitatZeroUnitTests"
time="0.001"/>
```

TC-FE-04 — Avanço de fase de planta: sequência correta e limite final

O objetivo do teste é validar se o frontend consegue representar de forma fiel o ciclo de crescimento das plantas, respeitando a ordem das fases e garantindo que não haja avanço além do estágio final definido. Nesse cenário, cada leitura de sensor ou atualização de estado deve refletir a progressão natural da planta, desde a germinação até a maturidade, seguindo a sequência estabelecida no modelo. O teste assegura que o sistema não pule etapas nem permita que uma planta ultrapasse o limite máximo de fase, evitando inconsistências visuais ou lógicas na interface.

Campo	Detalhe
Cenário	Na tela de detalhe da estufa, o botão "Avançar Fase" deve percorrer a sequência correta de fases de crescimento e ser desabilitado quando a planta já está em COLHEITA (última fase).
Entrada	Chamadas a <code>proximaFase()</code> para cada valor do enum: SEMENTE, GERMINACAO, CRESCIMENTO, MATURACAO, COLHEITA.
Saída esperada	SEMENTE→GERMINACAO, GERMINACAO→CRESCIMENTO, CRESCIMENTO→MATURACAO, MATURACAO→COLHEITA, COLHEITA→null

Log de evidência:

```
<testcase name="TC-FE-04 avancar fase retorna proxima fase correta e null apos COLHEITA"
          classname="com.workwell.habitatzero.HabitatZeroUnitTests"
time="0.018"/>
```

TC-FE-05 — Buffer de histórico do Dashboard respeita limite de 20 entradas

O objetivo do teste é validar se o frontend mantém o controle adequado sobre a quantidade de registros exibidos no histórico de leituras. Nesse cenário, o sistema deve armazenar e mostrar apenas as últimas 20 entradas recebidas, descartando automaticamente as mais antigas quando novas leituras chegam. Esse comportamento garante que o Dashboard permaneça leve e responsivo, evitando sobrecarga visual e problemas de desempenho.

Campo	Detalhe
Cenário	O Dashboard mantém um <code>ArrayDeque</code> com as últimas 20 leituras para plotar os gráficos. Quando o buffer está cheio e uma nova leitura chega, a entrada mais antiga deve ser descartada para evitar crescimento ilimitado de memória.
Entrada	Buffer já preenchido com 20 entradas; nova leitura com valores 99.0 em todos os campos.
Saída esperada	Buffer continua com <code>size == 20</code> ; última entrada é a nova leitura (<code>temperatura=99.0</code>); primeira entrada não é mais a original de índice 0.

Log de evidência:

```
<testcase name="TC-FE-05 buffer de historico descarta entrada mais antiga ao
atingir limite de 20"
    classname="com.workwell.habitatzero.HabitatZeroUnitTests"
time="0.008"/>
```

5 IOT

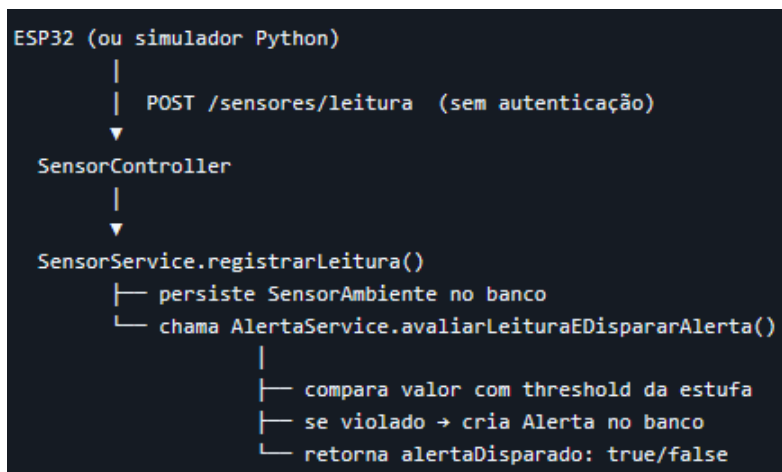
O módulo IoT é responsável pelo monitoramento contínuo das condições ambientais. A documentação descreve os sensores utilizados, oxigênio, umidade do solo, radiação e temperatura, e os thresholds configuráveis que definem os limites de alerta.

5.1 Lógica dos Sensores e Influência no Sistema

A lógica dos sensores e sua influência no sistema é o mecanismo que garante a interpretação correta dos dados coletados e a tomada de decisões automáticas dentro do projeto HabitatZero. Cada sensor físico possui thresholds previamente definidos que determinam limites aceitáveis de funcionamento. Quando uma leitura ultrapassa esses valores, a lógica implementada no microcontrolador ESP32 identifica o evento e aciona respostas adequadas, como a emissão de alertas, ajustes automáticos ou registros no histórico. Dessa forma, os sensores deixam de ser apenas dispositivos de coleta e passam a atuar como elementos ativos na governança da estufa, integrando-se de maneira estratégica ao fluxo de dados e às decisões críticas do projeto.

5.2 Visão Geral da Integração

A visão geral da integração descreve como os diferentes módulos e tecnologias do projeto se conectam para formar um sistema coeso e funcional. Essa integração envolve a comunicação contínua entre os sensores físicos, o microcontrolador ESP32, o firmware embarcado, o backend e a aplicação mobile, garantindo que os dados coletados sejam processados e disponibilizados em tempo real. O fluxo começa com a captura das variáveis ambientais pelos sensores, que são interpretadas pelo microcontrolador e transmitidas ao backend, onde são armazenadas e analisadas. Em seguida, essas informações são disponibilizadas para o aplicativo mobile, permitindo que os usuários acompanhem o estado da estufa e recebam alertas de severidade.



O campo `alertaDisparado` está presente na resposta do POST `/sensores/leitura`, permitindo que o ESP32 ou o simulador saiba imediatamente se a leitura gerou um alerta — sem precisar consultar um endpoint adicional.

Sensor 1 — Oxigênio (O₂)

É o sensor responsável por monitorar continuamente os níveis de oxigênio dentro da estufa do sistema HabitatZero, garantindo que o ambiente permaneça adequado para o crescimento das plantas e para a sobrevivência dos colonos.

Atributo	Valor
tipoSensor	OXIGENIO
Unidade	PERCENTUAL (%)
Threshold padrão	thresholdOxigenioMin = 19.5%
Range simulado	18,0% – 23,0%
Condição de alerta	valorLeitura < thresholdOxigenioMin

Lógica de severidade:

```
desvio = thresholdOxigenioMin - valorLeitura
desvio ≥ 5% → EMERGENCIA (ex: O2 = 14.5% com threshold 19.5%)
desvio ≥ 2% → CRITICO (ex: O2 = 17.0%)
desvio < 2% → ATENCAO (ex: O2 = 18.8%)
```


Influência no sistema: O oxigênio é o sensor mais crítico da colônia. Uma queda abaixo de 19,5% compromete a respiração humana e a sobrevivência das plantas. O sistema escalona automaticamente a severidade proporcionalmente ao desvio, permitindo resposta graduada — de atenção monitorada a evacuação de emergência.

Sensor 2 — Umidade do Solo

É o sensor responsável por medir a quantidade de água presente no substrato das plantas dentro da estufa do sistema HabitatZero, garantindo que o ambiente de cultivo mantenha condições ideais para o crescimento saudável.

Atributo	Valor
tipoSensor	UMIDADE_SOLO
Unidade	PERCENTUAL (%)
Threshold padrão	thresholdUmidadeMin = 30.0%
Range simulado	25,0% – 70,0%
Condição de alerta	valorLeitura < thresholdUmidadeMin

Lógica de severidade:

qualquer desvio abaixo do threshold → ATENCAO

Influência no sistema: Solo abaixo de 30% de umidade compromete a absorção de nutrientes pelas raízes. O alerta de ATENCAO sinaliza ao colono que é necessário acionar a irrigação manualmente pelo app (controle simulado) ou verificar o sistema de irrigação automática da estufa.

Sensor 3 — Radiação Externa

É o sensor responsável por monitorar os níveis de radiação que incidem sobre a estufa no sistema HabitatZero, garantindo que o ambiente interno permaneça seguro para o cultivo das plantas e para os colonos.

Atributo	Valor
tipoSensor	RADIACAO_EXTERNA
Unidade	MSV_HORA (mSv/h)
Threshold padrão	thresholdRadiacaoMax = 2.0 mSv/h
Range simulado	0,1 – 3,5 mSv/h
Condição de alerta	valorLeitura > thresholdRadiacaoMax

Lógica de severidade:

valor $\geq$ 3×threshold	→	EMERGENCIA	(ex: 6.0 mSv/h)
valor $\geq$ 2×threshold	→	CRITICO	(ex: 4.0 mSv/h)
valor > threshold	→	ATENCAO	(ex: 2.5 mSv/h)

Influência no sistema: Em ambientes lunares e marcianos, picos de radiação cósmica são previsíveis, mas fatais em doses acumuladas. A severidade em múltiplos do threshold reflete que o perigo cresce de forma não-linear: 2× o limite seguro já representa risco direto ao DNA celular das plantas e à saúde dos colonos.

Sensor 4 — Temperatura Interna

É o sensor responsável por monitorar continuamente o clima dentro da estufa do sistema HabitatZero, garantindo que as condições térmicas sejam adequadas tanto para o cultivo das plantas quanto para o conforto dos colonos.

Atributo	Valor
tipoSensor	TEMPERATURA
Unidade	CELSIUS (°C)
Threshold padrão	thresholdTemperaturaMax = 40.0°C
Range simulado	20,0°C – 50,0°C
Condição de alerta	valorLeitura > thresholdTemperaturaMax

Lógica de severidade:

qualquer leitura acima do threshold → CRITICO

Influência no sistema: Temperaturas acima de 40°C causam desnaturação de proteínas nas plantas e risco de hipertermia para os colonos. Diferente dos outros sensores, temperatura recebe sempre CRITICO, não existe "atenção" para superaquecimento dentro de uma estufa pressurizável, onde o calor acumulado escalona rapidamente.

5.3 Tabela Resumo — Thresholds e Severidades

Sensor	Condição	Severidade	Mensagem gerada
OXIGENIO	valor < min, desvio $\geq 5\%$	EMERGENCIA	"O ₂ abaixo do limite: X% (mínimo: Y%)"
OXIGENIO	valor < min, desvio $\geq 2\%$	CRITICO	"O ₂ abaixo do limite: X% (mínimo: Y%)"
OXIGENIO	valor < min, desvio < 2%	ATENCAO	"O ₂ abaixo do limite: X% (mínimo: Y%)"
UMIDADE_SOLO	valor < min	ATENCAO	"Umidade do solo crítica: X% (mínimo: Y%)"
RADIACAO_EXTERNA	valor $\geq 3 \times$ max	EMERGENCIA	"Radiação externa elevada: X mSv/h (máximo: Y mSv/h)"
RADIACAO_EXTERNA	valor $\geq 2 \times$ max	CRITICO	"Radiação externa elevada: X mSv/h (máximo: Y mSv/h)"
RADIACAO_EXTERNA	valor > max	ATENCAO	"Radiação externa elevada: X mSv/h (máximo: Y mSv/h)"
TEMPERATURA	valor > max	CRITICO	"Temperatura acima do limite: X°C (máximo: Y°C)"

Personalização por Estufa

Todos os thresholds são configuráveis individualmente por estufa via `PUT /estufas/{id}`. O app Android expõe isso na tela Controle Climático — o colono pode ajustar `thresholdTemperaturaMax` e `thresholdUmidadeMin` com sliders, e a mudança é imediatamente refletida na avaliação das próximas leituras IoT.

5.4 Implementação com Hardware Físico

O projeto inclui o firmware real em **IoT/FirmwareReal.cpp**, escrito para o ESP32. Abaixo está a especificação completa do hardware necessário para substituir o simulador Python por um dispositivo físico.

5.5 Microcontrolador

O microcontrolador é o coração do sistema embarcado, responsável por coordenar a coleta, processamento e transmissão dos dados provenientes dos sensores. O microcontrolador escolhido foi o ESP32, devido à sua versatilidade, baixo consumo de energia e capacidade de comunicação sem fio integrada. Ele atua como intermediário entre os sensores ambientais, que monitoram variáveis como oxigênio, umidade do solo, radiação e temperatura, e o backend da aplicação, garantindo que as informações sejam enviadas em tempo real para análise e tomada de decisão. Além disso, o ESP32 permite a execução de lógicas de firmware que definem ciclos de leitura, thresholds configuráveis e protocolos de comunicação.

Componente	Modelo recomendado	Justificativa
MCU	ESP32 DevKit V1 (ou ESP32-WROOM-32)	Wi-Fi 802.11 b/g/n nativo, ADC de 12 bits (0–4095), suporta HTTP sem shields adicionais

5.6 Sensores Físicos

Variável medida	Sensor recomendado	Pino no ESP32	Saída elétrica	Observações
Oxigênio (O₂)	Grove Oxygen Sensor (ME2-O2-Φ20)	GPIO 34 (ADC1_CH6)	Analógica 0–3,3 V	Saída proporcional à concentração. Conversão: $O_2\% = (leitura / 4095.0) \times 25.0$ conforme datasheet
Umidade do Solo	Sensor Capacitivo de Umidade v2.0	GPIO 35 (ADC1_CH7)	Analógica 0–3,3 V	Capacitivo (não enferruja). Inversamente proporcional: solo seco → valor alto (~3200), solo molhado → valor baixo (~1500). Requer calibração por substrato
Radiação	Módulo Geiger-Müller SEN0463 (DFRobot)	GPIO 32 (ADC1_CH4)	Pulsos digitais / tensão analógica convertida	Fornecer taxa de dose em $\mu\text{Sv/h}$. Divisão por 1000 para converter para mSv/h antes de enviar à API
Temperatura	DS18B20 (sonda impermeável)	GPIO 4 (OneWire)	Digital (protocolo 1-Wire)	$\pm 0,5^\circ\text{C}$ de precisão, à prova d'água. Para uso em estufa, a versão impermeável é preferível ao DHT22

Por que GPIO34 e GPIO35? Esses pinos são somente entrada (input-only) no ESP32, o que os torna ideais para leitura analógica, não há risco de configuração acidental como saída que danificaria o ADC.

5.7 Lógica do Firmware (FirmwareReal.cpp)

O firmware executa um ciclo simples a cada 10 segundos:

1. **Lê** cada sensor via **analogRead()** ou protocolo 1-Wire;
2. **Converte** o valor bruto (0–4095 para ADC de 12 bits) para a unidade física usando a fórmula do datasheet de cada componente.
3. **Monta** o payload JSON com **ArduinoJson**.
4. **Envia** via **HTTPClient** ao endpoint **POST /sensores/leitura** da API.
5. **Verifica** o campo **alertaDisparado** na resposta — se **true**, acende um LED vermelho no hardware (feedback local imediato, independente do app).

```
6. // Trecho do loop principal (FirmwareReal.cpp)
7. void loop() {
8.     enviarLeitura("OXIGENIO",      lerSensorOxigenio(),    "PERCENTUAL");
9.     enviarLeitura("UMIDADE_SOLO",   lerSensorUmidade(),    "PERCENTUAL");
10.    enviarLeitura("RADIACAO_EXTERNA", lerSensorRadiacao(),  "MSV_HORA");
11.    enviarLeitura("TEMPERATURA",     lerSensorTemperatura(), "CELSIUS");
12.    delay(10000); // aguarda 10 segundos
13. }
```

A função `enviarLeitura()` monta o JSON, faz o POST e loga o resultado na Serial (115200 baud) para depuração com o Monitor Serial do Arduino IDE.